
Practical AI for Academics

Afternoon session — hands-on with your own materials

Claes Bäckman

Leibniz Institute for Financial Research SAFE

Three messages: ground the agent in *your* files, run it as a critic before you publish, and keep verification close.

1 From mental model to your draft

The morning was tools and theory — **this afternoon we point them at your own paper, prose, and slides.**

Implication: the leverage is in materials only you have.

2 Critic, not co-author

Six subagents reading one draft beat one diffuse pass. The skill is **cheap pre-flight checking**, not replacing the referee.

Implication: use AI to find what you would miss.

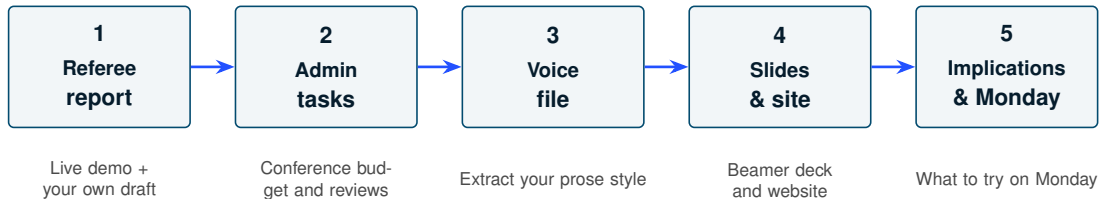
3 The bottleneck is judgement

Producing draft prose, slides, and websites is now cheap. **Reading, checking, and disagreeing** is where the researcher's edge sits.

Implication: verification is the skill to invest in.

Key takeaway. By the end of the session you will have run a referee report on your own draft, built a `voice.md`, and seen slides and a website generated from your materials.

Afternoon agenda — five hands-on blocks, ending with implications.



What you walk out with. A referee report on a draft you actually care about, a `voice.md` from your own writing, and a list of three things to try next week.

A referee report on your paper

Pick one draft you would like another pair of eyes on. Preferably a relatively close-to-finished draft

How to get started

Download the skill [at this url](#).

Place the skill in the Skill folder under the base Codex folder (a hidden folder under your user folder)

- To show hidden folders: Open File Explorer, click the View tab at the top, select Show, and check Hidden items

Restart VS Code

Skill should load by typing \$ skill-name

Live demo — `$review-paper` on a real draft, six subagents in parallel.

Setup

- One of my own papers, in its own folder
- `.tex` source, `.bib`, supporting materials
- One command: `$review-paper -target=AER`

Output

- A structured markdown report
- Referee-style comments, grouped by theme
- Specific line and section references

Six subagents running in parallel

- Contribution & positioning
- Identification & empirical strategy
- Robustness & threats to inference
- Writing & clarity
- Tables & figures
- Causal-language audit

Pattern: *six narrow critics beat one diffuse one.*

Key takeaway. The skill is a ~200-line markdown file. You can read it, edit it, and download it yourself.

Hands-on (20 min) — run a referee report on your own writing.

1. Open one of your own drafts in VS Code
2. Make sure it is in markdown or LaTeX — convert if needed
3. Run `$review-paper-light` for a 1–2 minute version, or `$review-paper` for the full six-agent run
4. Read the output. It's not all correct!

Debrief questions

- What did the report catch that you would not have seen by yourself?
- What did it get wrong?
- Which subagent gave you the most useful feedback?

What the demo is meant to show

1. **Grounded output.** The report is about *your* draft, not a generic checklist
2. **Subagents are why this works.** Six narrow critics catch things one diffuse reader misses
3. **Not a replacement for a referee.** It is a pre-flight check before you submit
4. **Transparent.** The skill is a $\sim$ 200-line markdown file — fork it, modify it, make it yours

Key takeaway. The point is not to trust the report. The point is to make hard feedback cheap enough that you ask for it before submitting — not after the desk reject.

Other tools generate referee reports too

Tool	What it does	Pricing
Refine.ink	Referee-style report on a paper PDF	<i>Paid</i>
coarse.ink	Lightweight version of the same idea	<i>≈ free</i>
Referee 2	Paid referee-report tool	<i>Paid</i>
Stanford Agentic Reviewer	Open agentic reviewer	<i>Free</i>

Many more out there — new ones launch every few weeks.

Key takeaway. The advantage of running your own skill is that it lives next to your draft, knows your conventions, and you can edit what it checks.

Now launch another agent to critique the referee report you just generated — ask Codex to write the instructions.

Organising a conference

Live demo: Administrative tasks Codex can take off your desk.

Setup

- A folder with a call for papers

The goal

- Build a budget for the conference

The output

- A budget spreadsheet with line items, totals, and assumptions

What we are using. Basic Codex commands — no skills, no MCP. Just the folder and the agent.

Two points the admin demo is meant to show

1. AI tools reduce the cost of administrative work.

- Not just for research
- Often the highest-leverage use case, because the friction is the highest and the quality bar is the lowest

2. Context is the difference between useful and useless.

- A chat tool gives you a generic conference-budget template
- Codex finds your keynote speakers, your dinner, and your PhD travel funding by reading the folder

Key takeaway. The same pattern — open the folder, hand the agent the materials, ask for the output — works for grant reports, course design, referee assignments, and committee memos.

Building a voice.md file

What are the three phrases you hate seeing in AI-edited prose?

Hands-on (15 min) — build a `voice.md` from your own writing.

We use a skill written by Mihail Velikov: `voice-extractor`.

1. Upload 4 PDFs of your own writing to a folder
2. Call the skill: `voice-extractor sample-papers`
3. Read through `voice.md` and verify — edit anything that does not sound like you

What goes in. Intros, abstracts, paragraphs you are proud of. 5–10k words is plenty.

What stays out. Co-authored sections where the prose is not yours. AI-edited drafts.

Key takeaway. Reference `voice.md` from `AGENTS.md`. Every future session loads your voice into context.

How to tell whether your `voice.md` is working

Signs it is working

- Edited paragraphs read more like you, less like a generic LLM
- Fewer em-dashes, fewer bullet lists, fewer “crucially” / “moreover”
- You stop having to say “preserve my voice” in every prompt
- Your coauthors stop noticing which paragraphs were AI-edited

Signs it is not

- Output still sounds generic
- AI tics persist after editing
- You keep adding “don’t use X” to every prompt

***Fix:** sample is too short, or includes too much AI-edited prose. Add more of your writing.*

Key takeaway. `voice.md` is a living file. Update it when your style shifts, or when an editor pushes you in a direction you want to keep.

SECTION 4

Slides and websites

Making slides with AI

The old path

- Open Beamer
- Write LaTeX by hand
- Fight the syntax
- Iterate slowly

Hours of mechanical work between idea and first draft.

The Codex path

- Open the project folder
- Hand Codex a file: *"Make a Beamer deck based on @paper.tex"*
- Compile, look at the output, iterate

This deck was built that way.

Key takeaway. This also means that we have to practice presentations in a different way

Including a style file gives Codex guidance to match the look you want.

Without a style file. Codex defaults to generic Beamer themes — bullet-heavy, blue-and-white, no action titles.

With a style file. The deck inherits your colour palette, frame structure, and title conventions.

How to build a `slides-style.md`

- Upload a few old presentations
- Ask Codex to analyse the structure, palette, and title style
- Save the result as `slides-style.md` in the project folder
- Reference it whenever you ask for a new deck

Key takeaway. The morning deck, this deck, and your future decks can all share one style file. Build it once.

Live demo — transcript and code → interactive website.

The task

- Take a folder of do-files and a transcript
- Build a standalone interactive website

What Codex does

1. Reads the code and the transcript
2. Sees how the code constructs the figures
3. Writes a standalone HTML page — one file, no dependencies

Why this works

- Codex knows how the figure was built — it has the code
- Codex knows how to write HTML
- Putting them together is exactly where agentic AI has an advantage over chat

***Pattern:** two skills the agent already has, glued together by the folder.*

Key takeaway. Same pattern works for policy briefs, replication packages, course websites, and conference programmes.

Implications and getting started

Separate publishing from research quality

Publishing gets noisier

- Journals fill with polished, competent-looking mediocrity
- Signal-to-noise drops and reviewing gets harder
- Editors and referees absorb the cost

Research can still get better

- Ideas that could not be written before reach readers
- The language barrier falls
- Feedback, formerly a privilege of elite departments, becomes universal

Key takeaway. The open question is whether we update how we **evaluate** research now that competent-looking writing is free. Because research is about to get better.

What this means for academic work

- **Friction drops an order of magnitude** in writing, editing, and admin
- **The bottleneck shifts** from producing to judging
- **Verification becomes the differentiator** — not typing speed, not LaTeX fluency
- **Disclosure norms are forming** — expect them to harden over the next year
- **Tooling will keep moving** — the principles (specificity, iteration, verification) will not

Key takeaway. The researchers who get the most out of these tools are not the most technical. They are the ones who treat AI as a colleague to argue with.

Verification is going to be increasingly important (and in the end also automated?)

Check	What to look for	How to do it
Run the code	Codex writes plausible Stata / R / Python that does not always run.	Run the code yourself.
Check the numbers	Statistics, dates, quotes get invented.	Re-derive from the table or paper.
Read the citations	Citations to papers that do not exist are common.	Check if you can find it on Google Scholar.
Re-read the passage	Codex confidently describes text not in the draft.	Open the file, read the section in question.

Key takeaway. You are faster with AI, not absent.

Disclosure and data norms are still in flux.

Disclosure

- **Journals** mostly require acknowledging AI assistance but specifics vary
- **Policy briefs, teaching, internal docs** have no strong norm yet (?)
- **Default rule:** disclose where your name is on it

Data handling

- Agentic tools run *locally*, but prompts and file excerpts may be sent to the model provider
- **Do not** paste confidential microdata, HR-sensitive material, or DUA-covered material into any AI tool without checking institutional policy
- Web chat $\neq$ local agent — different tools, different policies

Key takeaway. Check before pasting. Once is enough — the policy almost never changes mid-project.

Three things to try on Monday — and three habits to keep.

Three things to try Monday

1. Open your current project in VS Code. Ask Codex to draft `AGENTS.md`, then edit it.
2. Run `$review-paper-light` on a draft you are stuck on. Read the output critically.
3. Build a `voice.md` from 5–10k words of your own writing. Reference it from `AGENTS.md`.

Three habits to keep

- **Always run, re-read, verify.**
- **Preserve your voice.** Push back when the edit sounds generic.
- **Ask for critique, not flattery.** “Be harsher” is a valid instruction.

Key takeaway. A one-hour Monday investment in `AGENTS.md` and `voice.md` compounds across every later session.

Q & A

*Open floor. Email: claes.backman@gmail.com |
Guide: claesbackman.com/codex-vscode-guide*

References I

Appendix — AGENTS.md starter template.

About me

I am a researcher in [field].

How I want you to work with me

- Ask clarifying questions before generating long output.
- Critical, skeptical tone in feedback. Do not flatter.
- When editing, preserve my voice (see voice.md). No generic LLM phrasing.
- Avoid bullet lists and passive voice in formal writing.
- Cite the specific line or section you are commenting on.

Things to avoid

- Do not fabricate citations.
- Do not over-claim causality.
- Do not insert emoji or markdown decorations in formal docs.

Appendix — instructor's skill library.

Skill	Use
<code>\$review-paper</code>	Pre-submission referee report, 6 agents
<code>\$review-paper-light</code>	Fast 2-agent version, $\approx$ 1 minute
<code>\$review-paper-code</code>	Paper + code alignment review
<code>\$review-grant</code>	Pre-submission grant review, 6 agents
<code>\$review-pap</code>	Pre-analysis-plan review
<code>\$paper-version</code>	Paper $\rightarrow$ policy brief / 1-page / 5-page
<code>\$pdf-to-markdown</code>	Convert PDFs to readable markdown
<code>\$done</code>	Session capture and log

All in the starter folder. Fork them, modify them, make them yours.

Appendix — file format quick reference.

Format	Handling
<code>.md</code> , <code>.tex</code> , <code>.txt</code> , <code>.bib</code>	Native — no conversion
<code>.py</code> , <code>.R</code> , <code>.do</code> , <code>.ipynb</code>	Native — read, edit, run
<code>.csv</code>	Native up to a few thousand rows
<code>.docx</code>	Convert: <code>pandoc file.docx -o file.md</code>
<code>.xlsx</code>	Export to CSV first
<code>.dta</code> , <code>.rds</code>	Script-based conversion to CSV
PDF (born-digital, short)	Native; expect table mangling
PDF (long, scanned, equations)	Convert to markdown first

Appendix — signs of session drift.

- Codex ignores AGENTS.md
- Repeats mistakes you already corrected
- Invents files that do not exist
- Gives circular answers
- Forgets the last three turns

Fix. Start a fresh session or close and reopen. Re-state the task. Re-attach the relevant file.

30+ turns or the task has shifted → start fresh. Always.